THE UNIVERSITY OF HUDDERSFIELD
School of Computing and Engineering

CMI3509 Assignment II

Design and Demonstration of Relational and Graph Database Systems for Real-World Applications

Name: William West

Student ID: 2580784314

Date: 01/05/2026

1 Introduction

Database management systems provide different approaches for storing and analysing data, depending on the structure and relationships within the database. Relational databases are useful when data is structured, and consistency is required, whereas graph databases are more suitable when relationships between entities are central to the application domain (Stonebraker, 2010).

This report provides tutorials for two database systems through real-world use cases. PostgreSQL is applied to an Electronic Health Record (EHR) system to demonstrate relational modelling, structured querying, and ACID-compliant transactions (PostgreSQL Global Development Group, 2024). Neo4j is then used in a banking fraud detection scenario to demonstrate graph traversal and pattern analysis with Cypher queries (Neo4j, Inc., 2024).

2 Electronic Health Record System (PostgreSQL)

Electronic Health Record (EHR) systems store health information, including patient records, clinician assignments, and appointment histories. These systems required accurate and consistent records for patient safety.

PostgreSQL is a suitable Database management system for an EHR system, as healthcare data is highly structured. Strong consistency is also provided. In this instance, entities such as patients, clinicians, and appointments are relational tables, with the appointment table acting as a link. Also, ACID-compliant transactions result in reliable updates if multiple clinicians may access the system simultaneously PostgreSQL Global Development Group, 2024).

2.1 Database Design

2.1.1 Schema Creation

The database schema is created with CREATE TABLE statements for the three entities: patient, clinician, and appointment. Each table contains a primary key, while relationships between tables are enforced through foreign keys.

Figure 1 shows the SQL schema used to create the database structure. The full implementation is provided in Appendix 6.1.

Figure 1

PostgreSQL schema creation statements

```
CREATE TABLE patient (
    patient_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    date_of_birth DATE NOT NULL,
    phone_number VARCHAR(20) NOT NULL
);

CREATE TABLE clinician (
    clinician_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    speciality VARCHAR(100) NOT NULL
);

CREATE TABLE appointment (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL REFERENCES
patient(patient_id),
    clinician_id INT NOT NULL REFERENCES
clinician(clinician_id),
    appointment_date TIMESTAMP NOT NULL,
    reason VARCHAR(255) NOT NULL
);
```

2.1.2 Explanation of Key Concepts

Table 1

Key SQL components used in the schema

SQL Component	Description
SERIAL PRIMARY KEY	Automatically generates a unique, incrementing identifier for each record in a table.
VARCHAR(n)	Stores variable-length text up to n characters.
NOT NULL	Ensures that a field must contain a value and cannot be left empty.
FOREIGN KEY (REFERENCES)	Links records between tables, enforcing relationships and data integrity.
TIMESTAMP	Stores both date and time values for appointments.

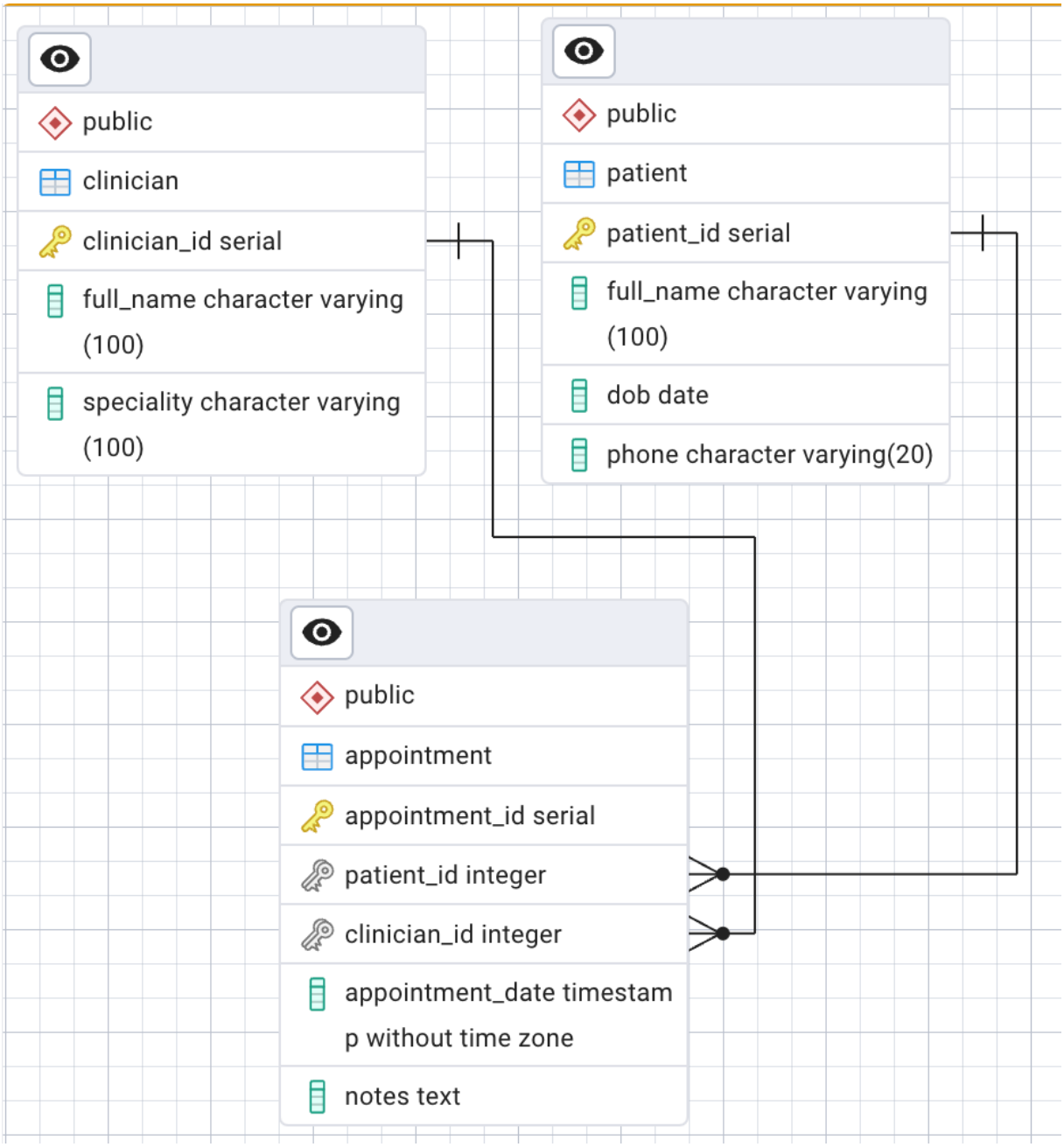
2.1.3 Entity Relationship Diagram (ERD)

Figure 2 presents the ERD for the EHR system. The appointment table is a linking entity between patients and clinicians. These relationships are one-to-many, as a single patient or clinician may be associated with several appointments.

Foreign key constraints enforce referential integrity, ensuring that appointment records always reference valid patients and clinicians (Stonebraker, 2010).

Figure 2

Entity Relationship Diagram for the EHR database



2.2 Data Implementation

2.2.1 Data Insertion

Sample data is inserted using INSERT statements to simulate healthcare records while maintaining relationships between tables.

Figure 3 shows example insertion queries.

Figure 3

SQL insertion statements for healthcare records

```
INSERT INTO patient (full_name, date_of_birth,
 phone_number)
VALUES
('John Smith', '1985-06-15', '07111111111'),
('Sarah Ahmed', '1992-09-21', '07222222222'),
('Michael Brown', '1978-03-08', '07333333333');

INSERT INTO clinician (full_name, speciality)
VALUES
('Dr Emily Carter', 'Cardiology'),
('Dr James Wilson', 'General Practice');

INSERT INTO appointment (patient_id, clinician_id,
 appointment_date, reason)
VALUES
(1, 1, '2025-04-15 10:00:00', 'Chest pain
 assessment'),
(2, 2, '2025-04-16 14:30:00', 'Routine health
 check'),
(3, 2, '2025-04-17 09:00:00', 'Follow-up
 consultation');
```

2.3 Querying the Database

2.3.1 Basic Queries

Basic SQL queries retrieve information from individual tables. The following query selects all records from the patient table.

Figure 4

Basic SQL query retrieving patient records

```
SELECT * FROM patient;
```

Figure 5 shows the resulting output.

Figure 5

Output of patient table query

	patient_id	full_name	date_of_birth	phone_number
1	1	John Smith	1985-06-15	07111111111
2	2	Sarah Ahmed	1992-09-21	07222222222
3	3	Michael Brown	1978-03-08	07333333333

2.3.2 Multi-table Queries (JOINs)

Multi-table queries are essential to match and combine related tables from several tables. This example uses aliases that act as variables to simplify references, with a, p, and c representing the appointment, patient, and clinician tables, respectively.

Although foreign keys define relationships within the schema, JOIN conditions must still be specified in queries. The ON clause determines how rows from different tables are matched.

Figure 6 shows the query.

Figure 6

SQL JOIN query combining appointment, patient, and clinician data

```
SELECT
    p.full_name AS patient_name,
    c.full_name AS clinician_name,
    c.speciality,
    a.appointment_date,
    a.reason
FROM appointment a
JOIN patient p ON a.patient_id = p.patient_id
JOIN clinician c ON a.clinician_id = c.clinician_id;
```

This query combines patient, clinician, and appointment data using JOIN operations to retrieve related records efficiently.

Figure 7 shows the resulting output.

Figure 7

Output of multi-table JOIN query

patient_name	clinician_name	speciality	appointment_date	reason
John Smith	Dr Emily Carter	Cardiology	2025-04-15 10:00:00	Chest pain assessment
Sarah Ahmed	Dr James Wilson	General Practice	2025-04-16 14:30:00	Routine health check
Michael Brown	Dr James Wilson	General Practice	2025-04-17 09:00:00	Follow-up consultation

2.4 Transactions and ACID Compliance

2.4.1 Transaction Rollback (Atomicity)

Transactions ensure that groups of database operations are executed together, meaning either all transactions are successful or all transactions fail. (PostgreSQL Global Development Group, 2024).

The following example demonstrates a transaction rollback.

Figure 8

Transaction rollback example demonstrating atomicity

```
BEGIN;

INSERT INTO patient (full_name, date_of_birth,
 phone_number)
VALUES ('Test Patient', '1990-01-01', '07000000000');

-- Fails; invalid patient_id (foreign key violation)
INSERT INTO appointment (patient_id, clinician_id,
 appointment_date, reason)
VALUES (999, 1, '2025-05-01 10:00:00', 'Test
 appointment');

ROLLBACK;
```

In this example, the first statement executes successfully. However, the second statement violates a foreign key constraint because the referenced patient_id does not exist. Because both statements are enclosed within a transaction using BEGIN, the ROLLBACK command reverses all changes.

2.4.2 Constraint Violation (Consistency)

Consistency ensures that the database remains in a valid state by enforcing constraints such as NOT NULL and foreign key relationships.

The following example demonstrates a constraint violation.

Figure 9

Constraint violation example

```
INSERT INTO appointment (patient_id, clinician_id,
 appointment_date, reason)
VALUES (1, NULL, '2025-05-01 11:00:00', 'Routine
 check');
```

This query fails because the clinician_id column is defined with a NOT NULL constraint.

Figure 10

PostgreSQL error generated by NOT NULL constraint violation

```
Query 1 ERROR at Line 1: : ERROR:  null value in column
"clinician_id" of relation "appointment" violates not-null
constraint
```

3 Banking Fraud Detection System (Neo4j)

3.1 Use Case Overview

Modern banking systems process large volumes of financial transactions between accounts, making fraudulent behaviour difficult to identify using traditional approaches alone. Fraud often involves hidden patterns within networks of accounts and transactions, including repeated transfer chains or circular movement of funds (Karim et al., 2023).

Neo4j is well-suited to this application because it models data as nodes and relationships rather than relational tables. Relationships are stored directly and traversed efficiently rather than reconstructed through joins (Angles & Gutierrez, 2008).

Neo4j uses the Cypher query language, which is designed specifically for graph traversal and pattern matching (Neo4j, Inc., 2024).

3.2 Graph Database Design

3.2.1 Node Creation

The primary entities in the graph, including people and bank accounts, are created as nodes using the CREATE keyword.

Figure 11 shows the node creation query. The full implementation is provided in Appendix 6.2.

Figure 11

Neo4j node creation statements

```
// Create people
CREATE (:Person {name: 'Alice'});
CREATE (:Person {name: 'Bob'});
CREATE (:Person {name: 'Charlie'});

// Create accounts
CREATE (:Account {id: 101});
CREATE (:Account {id: 102});
CREATE (:Account {id: 103});
CREATE (:Account {id: 104}); // extra account for Alice
```

While CREATE always generates new nodes, Neo4j also provides the MERGE keyword, which creates a node only if it does not already exist (Neo4j, Inc., 2024).

Figure 12 shows an example using MERGE.

Figure 12

Using MERGE to prevent duplicate nodes

3.2.2 Relationship Creation

Relationships are created by matching existing nodes and defining directional relationships between them.

Figure 13

Neo4j relationship creation query

```
MATCH (alice:Person {name: 'Alice'})
MATCH (a101:Account {id: 101})
MATCH (a104:Account {id: 104})
MERGE (alice)-[:OWNS]->(a101)
MERGE (alice)-[:OWNS]->(a104);
```

It is best practice to place each MATCH clause on a separate line to improve readability and reduce Cartesian product warnings.

A Cartesian product occurs when multiple MATCH clauses return several nodes independently, causing Neo4j to combine all possible pairs of results.

Figure 14 shows an example warning.

Figure 14

Neo4j Cartesian product warning

03N90: Cartesian product

The disconnected pattern '(p:Person {name: 'Alice'}), (a:Account {id: 101})' builds a cartesian product. A cartesian product may produce a large amount of data and slow down query processing.

```
MATCH (p:Person {name: 'Alice'}), (a:Account {id: 101})
^
MERGE (p)-[:OWNS]→(a);
```

3.2.3 Explaining Cypher

Table 2

Fundamental Cypher syntax

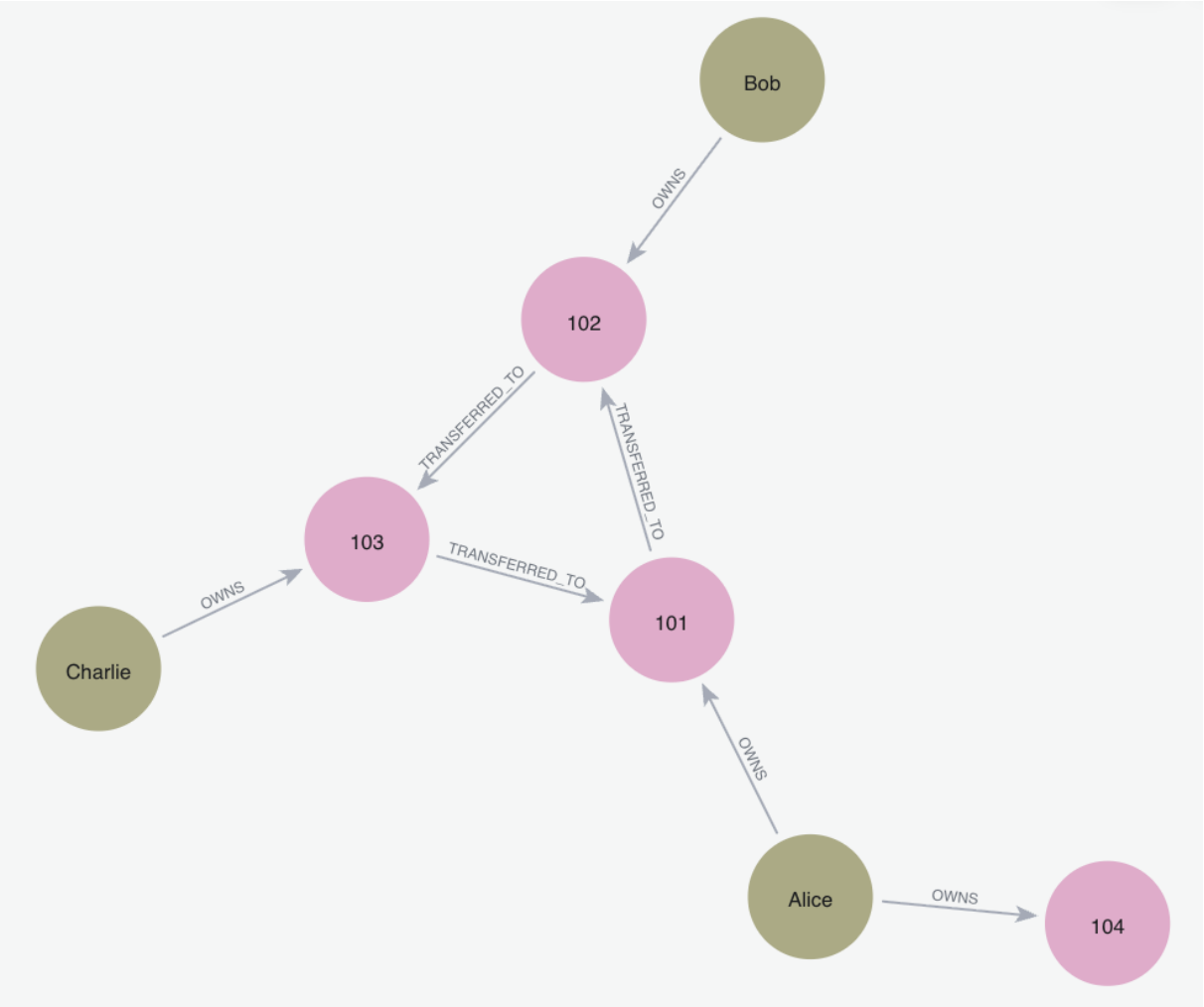
Cypher Syntax	Description
//	Denotes a comment.
() (Parentheses)	Represents a node in a graph.
:Label	Defines a label, grouping nodes of the same type.
(p:Person)	Uses an alias, p, to refer to a matched node later in the query.
{key: value}	Properties stored as key–value pairs.
[] (Square Brackets)	Represents a relationship.
–[:TYPE]–>	A directed relationship between nodes.
<–[:TYPE]–	A relationship in the opposite direction.
;	Marks the end of a query.

3.2.4 Graph Structure Diagram

Figure 15 illustrates the structure of the banking fraud detection graph.

Figure 15

Graph structure for the fraud detection system



3.3 Querying the Graph

3.3.1 One-step Traversal

One-step traversals identify directly connected nodes. The following query finds all accounts owned by Alice.

Figure 16

One-step traversal query

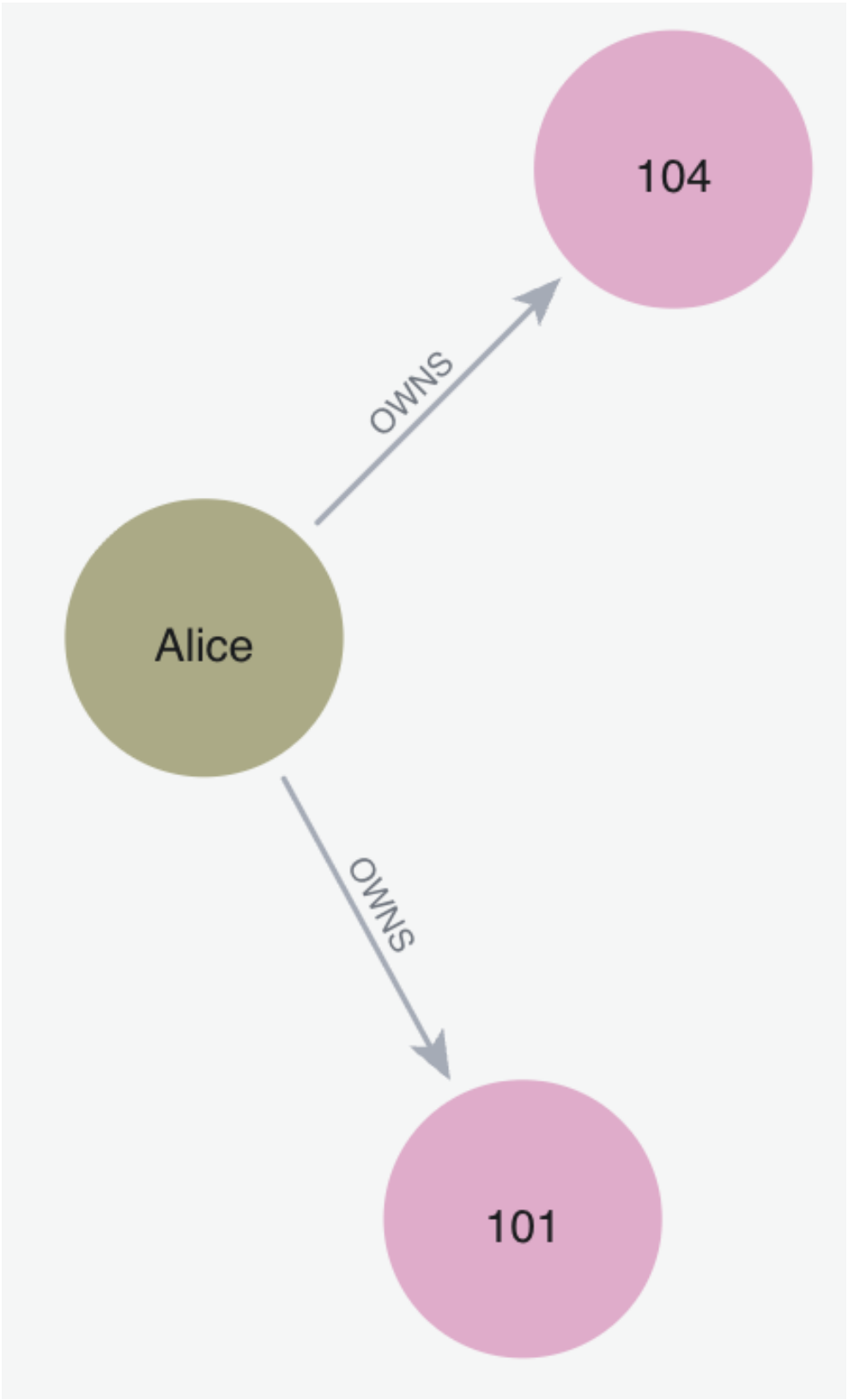
```
MATCH (p:Person {name: 'Alice'})-[o:OWNS]->(a:Account)
RETURN p, o, a;
```

This query begins at the Person node representing Alice and follows the OWNS relationship to connected accounts.

Figure 17 shows the output.

Figure 17

Output of one-step traversal query



The query can then be extended to identify accounts receiving transfers from Alice's accounts.

Figure 18

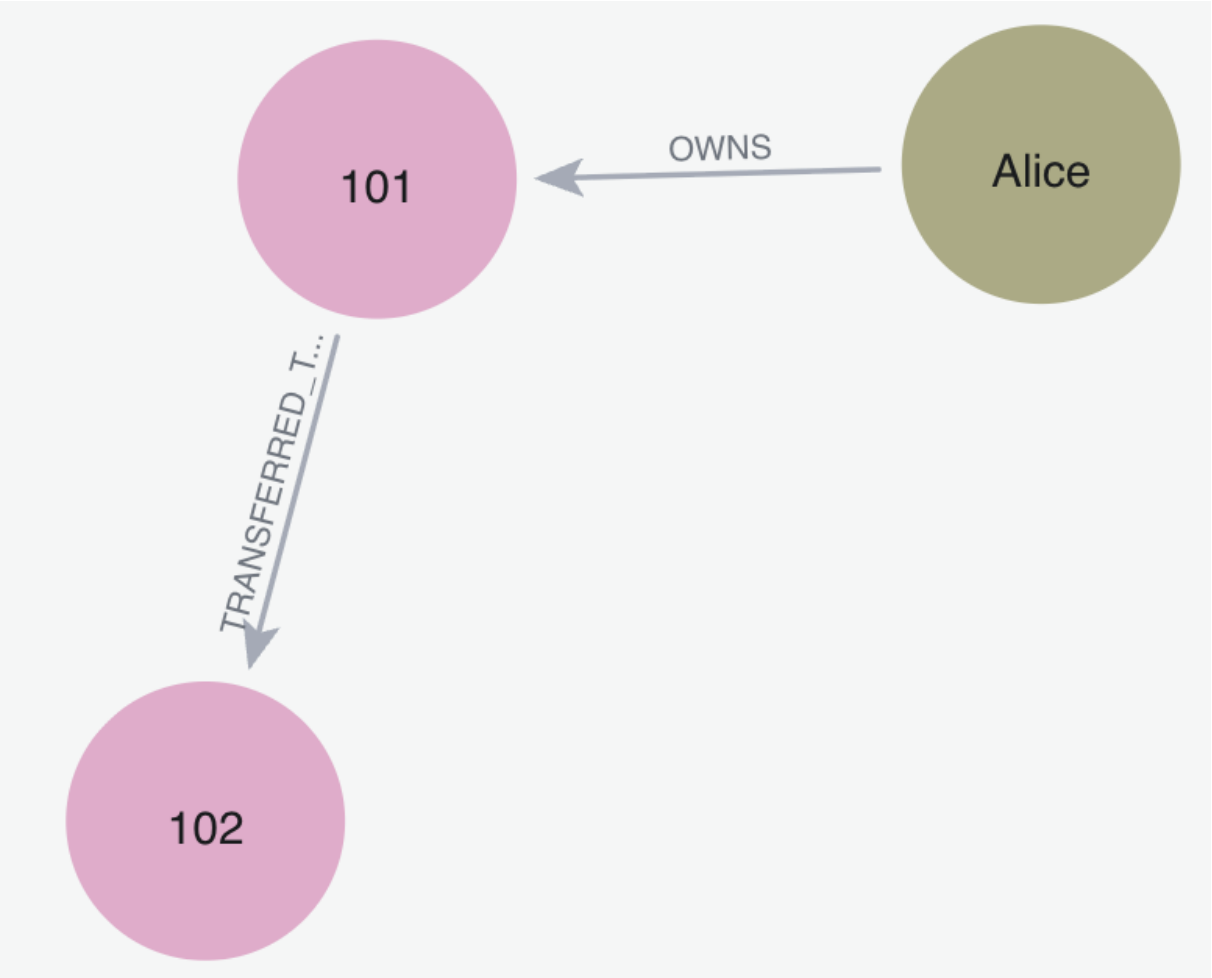
Extended traversal query following transfer relationships

```
MATCH (p:Person {name: 'Alice'})-[o:OWNS]->(a:Account)
MATCH (a)-[t:TRANSFERRED_TO]->(target:Account)
RETURN p, o, a, t, target;
```

Figure 19 shows the resulting graph.

Figure 19

Output of extended traversal query



3.3.2 Two-step Traversal

Two-step traversals identify indirect connections between accounts.

Figure 20

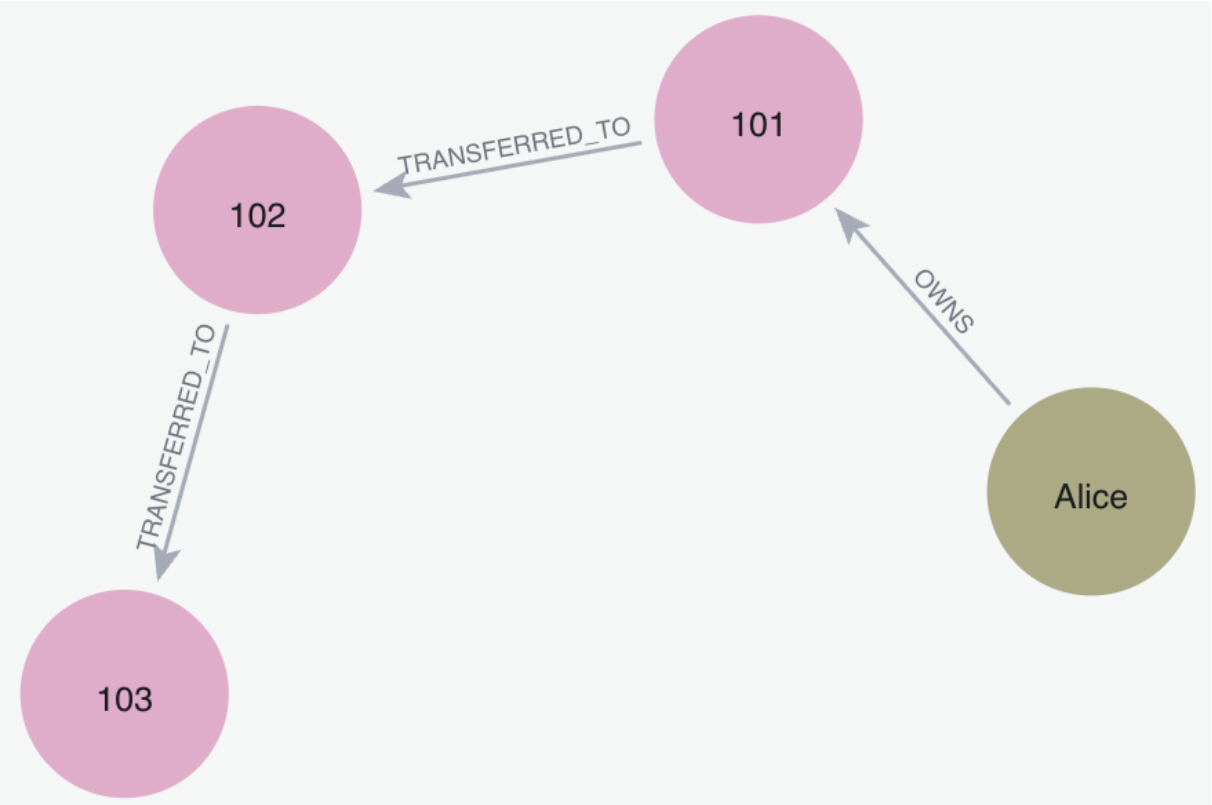
Two-step traversal query

```
MATCH (p:Person {name: 'Alice'})-[d:OWNS]->(a:Account)
      -[e:TRANSFERRED_TO]->(b:Account)
      -[f:TRANSFERRED_TO]->(c:Account)
RETURN p, a, b, c, d, e, f;
```

This query follows two consecutive TRANSFERRED_TO relationships, forming a transaction chain spanning two transfer steps.

Figure 21

Output of two-step traversal query



3.3.3 Variable-length Traversal

Cypher also supports variable-length traversals when the number of transfer steps is unknown.

Figure 22

Variable-length traversal query

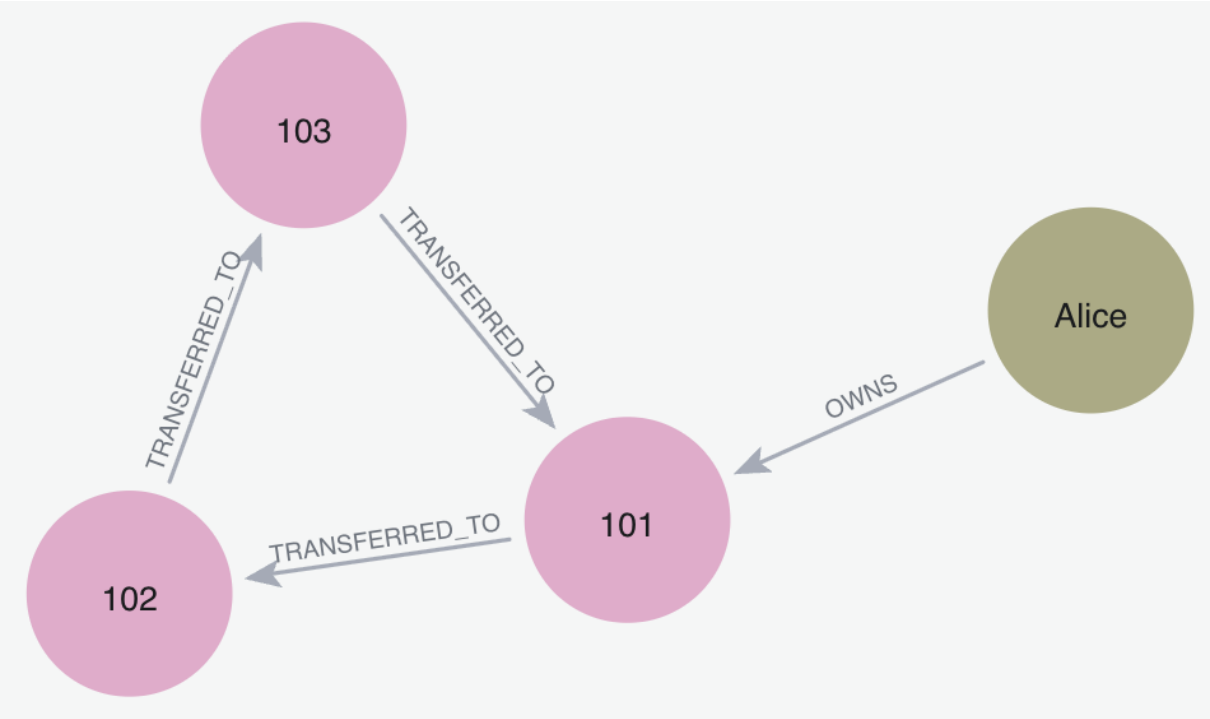
```
MATCH path = (p:Person {name: 'Alice'})-[:OWNS]->(:Account)
             -[:TRANSFERRED_TO*1..3]->(:Account)
RETURN path;
```

In this query, the '1..3' notation instructs Neo4j to follow between one and three TRANSFERRED_TO relationships. The expression 'path =' assigns the traversal to a variable, allowing the full sequence of nodes and relationships to be returned as a single result.

Figure ***23*** shows the resulting paths.

Figure 23

Output of variable-length traversal query



3.4 Fraud Detection Using Patterns

Graph databases are effective for fraud detection because suspicious behaviour often appears as patterns of relationships rather than isolated transactions (Karim et al., 2023).

3.4.1 Detecting Circular Transfer Patterns

The following query identifies circular transaction paths between accounts.

**bit more detail

Figure 24

Circular transaction detection query

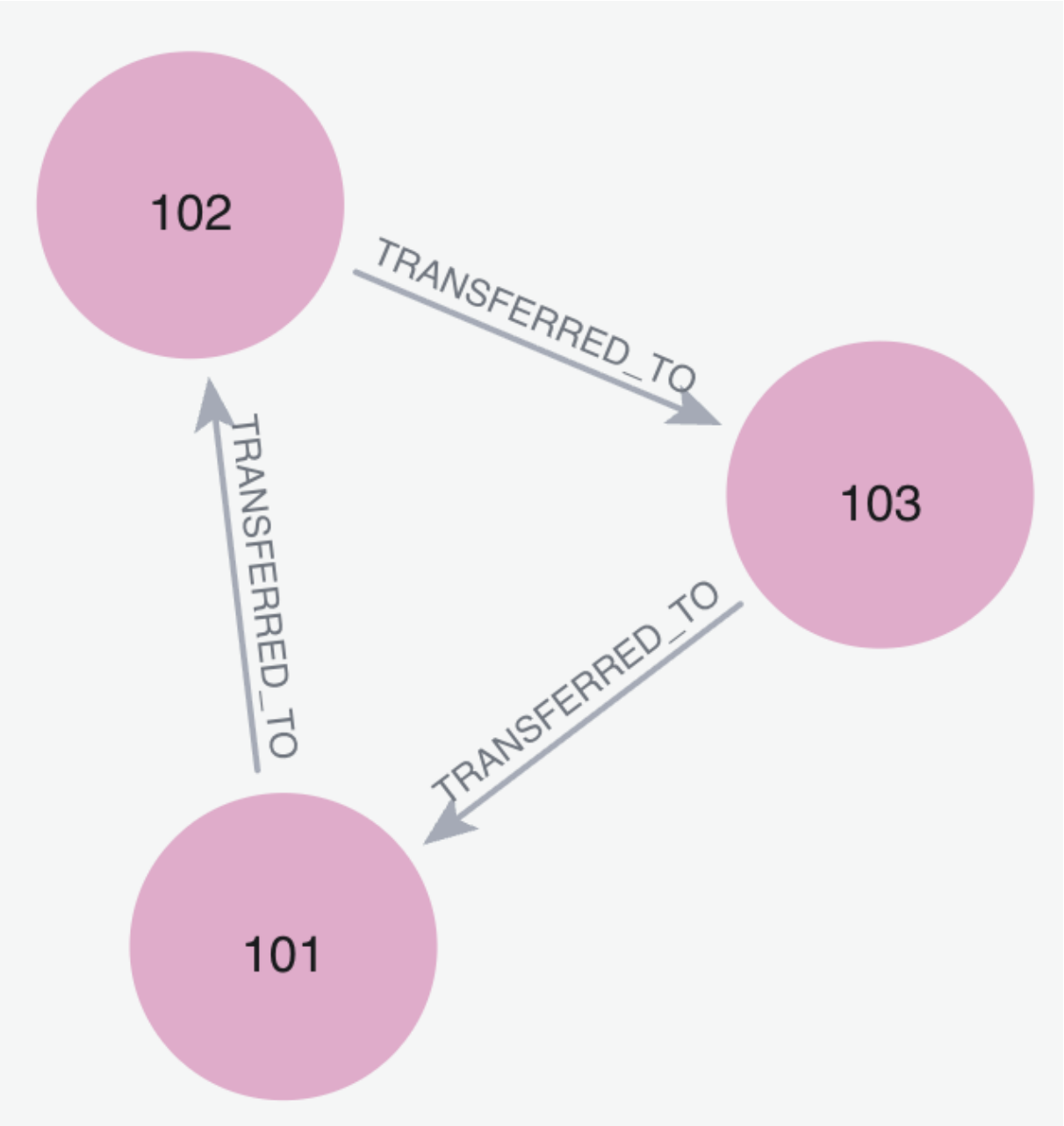
```
MATCH path = (a:Account)-[:TRANSFERRED_TO*2..5]->(a)
RETURN path;
```

This query searches for paths where funds are transferred through several accounts before eventually returning to the original account.

Figure 25 shows the resulting graph structure.

Figure 25

Output of circular transaction detection query



Although the graph visualisation displays a single cycle, Neo4j may return multiple matching paths because the same cycle can be identified from different starting nodes.

3.4.2 Interpreting Circular Patterns

Circular transaction patterns may indicate attempts to obscure the movement of funds. However, the existence of a circular pattern does not necessarily confirm fraudulent behaviour. Instead, it highlights accounts and transactions that may require further investigation.

In real-world systems, additional factors such as transaction timing, frequency, and customer behaviour would also be analysed.

3.4.3 Ranking Indirect Connections

Graph databases can also measure how strongly accounts are connected through indirect transfers. The following query identifies accounts repeatedly reached through two-step transaction paths originating from Alice's accounts.

Figure 26

Query ranking indirect account connections

```
MATCH (p:Person {name: 'Alice'})-[:OWNS]->(a:Account)
      -[:TRANSFERRED_TO]->(middle:Account)
      -[:TRANSFERRED_TO]->(target:Account)
WHERE a <> target
RETURN target.id AS Account,
       count(*) AS Strength
ORDER BY Strength DESC;
```

This query traverses from Alice to indirectly connected accounts through two consecutive TRANSFERRED_TO relationships.

Unlike previous traversal examples, this query introduces aggregation using count(*). The count(*) function counts matching traversal paths to each target account. This value is returned as Strength, representing how frequently an account appears within indirect transaction chains.

The WHERE a <> target condition excludes Alice's original accounts, while ORDER BY Strength DESC sorts the strongest indirect connections first.

Figure 27 shows the query output.

Figure 27

Output ranking indirect account connections

Account	Strength
103	1

3.5 Professional Considerations in Database Systems

Highly sensitive data is involved when dealing with healthcare and banking systems, therefore professional and ethical considerations are important when designing database systems.

Electronic Health Records systems need to ensure patient confidentiality, as well as secure access and data integrity. On the other hand, fraud detection systems must balance effective monitoring with tentative use of financial data – incorrectly identifying legitimate behaviour as fraudulent may harm customers.

These considerations demonstrate that database engineering involves both technical implementation and responsible management of sensitive information.

4 Conclusion

This report presented tutorials demonstrating the design and use of both relational and graph database systems through real-world applications.

PostgreSQL was used to model an Electronic Health Record system, where structured schemas, referential integrity, and ACID-compliant transactions ensured reliable management of healthcare data. Neo4j was then applied to banking fraud detection, where graph traversal and relationship-based querying enabled suspicious transaction patterns to be analysed efficiently.

Overall, the report demonstrated that different database models are suited to different forms of data and analysis. Relational databases remain effective for structured transactional systems, whereas graph databases provide advantages when relationships and connected patterns are central to the application domain.

References

Angles, R., & Gutierrez, C. (2008). Survey of graph database models. *ACM Computing Surveys, 40*(1), 1–39.

Karim, M. R., Islam, M. A., & Rahman, M. (2023). Graph-based fraud detection in financial transaction networks. *Journal of Financial Crime, 30*(2), 455–470.

Neo4j, Inc. (2024). *Neo4j Cypher manual*. https://neo4j.com/docs/cypher-manual/current/

PostgreSQL Global Development Group. (2024). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Stonebraker, M. (2010). SQL databases v. NoSQL databases. *Communications of the ACM, 53*(4), 10–11.

5 Appendix

5.1 SQL Code

```sql
-- 1. Create patient table
CREATE TABLE patient (
    patient_id SERIAL PRIMARY KEY,
    full_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    phone_number VARCHAR(20)
);

-- 2. Create clinician table
CREATE TABLE clinician (
    clinician_id SERIAL PRIMARY KEY,
    full_name VARCHAR(50) NOT NULL,
    speciality VARCHAR(100) NOT NULL
);

-- 3. Create appointment table
CREATE TABLE appointment (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL,
    clinician_id INT NOT NULL,
    appointment_time TIMESTAMP NOT NULL,
    reason VARCHAR(255),

    FOREIGN KEY (patient_id) REFERENCES patient(patient_id),
    FOREIGN KEY (clinician_id) REFERENCES clinician(clinician_id)
);
```

```sql
-- 4. Insert sample patients
INSERT INTO patient (first_name, last_name, date_of_birth, phone_number)
VALUES
('Alice Brown', '1985-04-12', '07123456789'),
('James Wilson', '1992-09-23', '07234567890'),
('Sarah Taylor', '1978-01-30', '07345678901');

-- 5. Insert sample clinicians
INSERT INTO clinician (first_name, last_name, speciality)
VALUES
('Emily Smith', 'General Practitioner'),
('Robert Jones', 'Cardiology'),
('Helen Clark', 'Dermatology');

-- 6. Insert sample appointments
INSERT INTO appointment (patient_id, clinician_id, appointment_time, reason)
VALUES
(1, 1, '2026-04-28 09:30:00', 'General check-up'),
(2, 2, '2026-04-28 11:00:00', 'Chest pain assessment'),
(3, 3, '2026-04-29 14:15:00', 'Skin irritation'),
(1, 2, '2026-04-30 10:00:00', 'Follow-up appointment');

-- 7. Basic query: retrieve all patients
SELECT * FROM patient;
```

5.2 Neo4j Code

```
-- 8. Multi-table query using JOINs
SELECT
    p.full_name AS patient_name,
    c.full_name AS clinician_name,
    c.speciality,
    a.appointment_date,
    a.reason
FROM appointment a
JOIN patient p
    ON a.patient_id = p.patient_id
JOIN clinician c
    ON a.clinician_id = c.clinician_id,


-- 9. Transaction rollback example
BEGIN;

INSERT INTO patient (full_name, date_of_birth, phone_number)
VALUES ('Temporary Patient', '1999-05-10', '07000000000');

-- This fails because patient_id 999 does not exist
INSERT INTO appointment (patient_id, clinician_id, appointment_time, reason)
VALUES (999, 1, '2026-05-01 12:00:00', 'Invalid appointment');

ROLLBACK;

ROLLBACK;

-- 10. Constraint violation example
-- This fails because clinician_id cannot be NULL
INSERT INTO appointment (patient_id, clinician_id, appointment_time, reason)
VALUES (1, NULL, '2026-05-02 13:00:00', 'Missing clinician');

-- 11. Check final appointment records
SELECT *
FROM appointment;
```

5.3 Neo4j Code

```
CREATE (:Person {name: 'Alice'});
CREATE (:Person {name: 'Bob'});
CREATE (:Person {name: 'Charlie'});

CREATE(:Account {id: 101});
CREATE(:Account {id: 102});
CREATE(:Account {id: 103});
CREATE(:Account {id: 104}); // extra
account for Alice

MERGE (:Person {name: 'Alice'});

// 3. Create ownership relationships
MATCH (alice:Person {name: 'Alice'})
MATCH (a101:Account {id: 101})
MATCH (a104:Account {id: 104})
MERGE (alice)-[:OWNS]->(a101)
MERGE (alice)-[:OWNS]->(a104);
```

```
// Cartesian product example
MATCH (p:Person {name: 'Alice'}),
(a:Account {id: 101})
MERGE (p)-[:OWNS]->(a);

//More relationships
MATCH (bob:Person {name: 'Bob'})
MATCH (a102:Account {id: 102})
MERGE (bob)-[:OWNS]->(a102);

MATCH (charlie:Person {name:
'Charlie'})
MATCH (a103:Account {id: 103})
MERGE (charlie)-[:OWNS]->(a103);

MATCH (a101:Account {id:101})
MATCH (a102:Account {id:102})
MATCH (a103:Account {id:103})
MERGE (a101)-[:TRANSFERRED_TO]->(a102)
MERGE (a102)-[:TRANSFERRED_TO]->(a103)
MERGE (a103)-[:TRANSFERRED_TO]->(a101);

// 7. One-step traversal: accounts owned by Alice
MATCH (p:Person {name: 'Alice'})-[o:OWNS]->(a:Account)
RETURN p, o, a;
```

```
// 8. Extended traversal
MATCH (p:Person {name: 'Alice'})-
[o:OWNS]->(a:Account)
MATCH (a)-[t:TRANSFERRED_TO]->
(target:Account)
RETURN p, o, a, t, target;

// 9. Two-step traversal
MATCH (p:Person {name: 'Alice'})-
[d:OWNS]->(a:Account)
  -[e:TRANSFERRED_TO]->(b:Account)
  -[f:TRANSFERRED_TO]->(c:Account)
RETURN p, a, b, c, d, e, f;
```

```
// 10. Variable-length traversal
MATCH path = (p:Person {name:
'Alice'})-[:OWNS]->(:Account)
             -[:TRANSFERRED_TO*1..3]->
(:Account)
RETURN path;

// 11. Detect circular transfer
patterns
MATCH path = (a:Account)-
[:TRANSFERRED_TO*2..5]->(a)
RETURN path;

// 12. Detect circular transfer
patterns with DISTINCT
MATCH path = (a:Account)-
[:TRANSFERRED_TO*2..5]->(a)
RETURN DISTINCT path;

// 13. Ranking indirect connections
MATCH (p:Person {name: 'Alice'})-
[:OWNS]->(a:Account)
      -[:TRANSFERRED_TO]->
(middle:Account)
      -[:TRANSFERRED_TO]->
(target:Account)
WHERE a <> target
RETURN target.id AS Account,
       count(*) AS Strength
ORDER BY Strength DESC;
```